

Guion de Exposicion

Integracion del Boleto NFT (ERC-721) portando el Lab 12

Documento guia para exponer: **que se cambio, que es cada cosa y que hace**, con la logica detallada de la integracion blockchain + NFT sobre el sistema de tickets de eventos. Cada seccion incluye un **guion hablado** (en cursiva) que puedes leer directamente al presentar.

Sistema	Tickets de eventos/conciertos con registro on-chain y boleto NFT
Backend	Node.js + Express + MariaDB + ethers v6 + Hardhat
Frontend	Next.js 16 + React 19 + ethers v6 + MetaMask
Blockchain	Ganache (red local) - Hardhat como toolchain en segundo plano
Contratos	Events.sol (registro) + EventoNFT.sol (ERC-721, OpenZeppelin 4.9.6)

Contexto: que teniamos y que se pidio

El proyecto **mod_02_productos** (backend) + **frontend_productos** (dashboard) es un sistema de **tickets de eventos**. Antes de estos cambios ya sabia:

- Guardar los datos legibles del evento **on-chain** mediante el contrato **Events.sol** (lugar, fecha, precio, aforo, organizador).
- Generar un **JSON de metadata** con estilo NFT por cada evento (metadata/<id>.json).
- Firmar transacciones desde el navegador con **MetaMask** (flujo del Lab de facturas).

Lo que faltaba: el sistema hablaba de "boleto NFT" pero **nunca acunaba un NFT real**. El Lab 12 (facturas) si tenia esa pieza: un contrato **ERC-721** (FacturaNFT) que acunaba un NFT por factura. La tarea fue **portar esa pieza** al mundo de eventos: crear el boleto NFT, sus endpoints, su verificacion y sus botones en el dashboard.

■ **GUION:** Nuestro sistema de tickets ya registraba los eventos en la blockchain y generaba su metadata, pero el "boleto NFT" era solo de nombre: no existia el token. Tomamos la parte de NFT del Lab 12 de facturas y la adaptamos a eventos, para que cada evento pueda acunarse como un NFT ERC-721 con dueño verificable.

El corazon de la logica: el mapeo Lab 12 -> Eventos

Lab 12 (facturas)	Este proyecto (eventos)	Rol
FacturaStorage.sol -> createFactura	Events.sol -> crearEvento (ya existia)	Registro de datos legibles on-chain
FacturaNFT.sol (ERC-721)	EventoNFT.sol (ERC-721) (nuevo)	Boleto NFT: propiedad + metadata + hash
mintFacturaNFT(id, cliente, monto, hash, to)	mintEventoNFT(id, name, lugar, precio, hash, to)	Acunar el NFT
facturaNFTService.js (web3.js)	eventoNftContract.js (ethers v6)	Verificacion backend (solo lectura)
facturas.js: mintNFT / verificarNFT	lib/nft.ts	Capa web3 del frontend
Botones MINT / VERIFICAR NFT	Botones ■ / ✓ en EventsTable.tsx	Interfaz de usuario

Dos diferencias tecnicas clave respecto al lab: (1) usamos **ethers v6** en vez de web3.js para unificar con el resto del proyecto; (2) el frontend pide la configuracion del NFT al backend en caliente (endpoint /nft/config), asi tras cada despliegue usa la nueva direccion sin editar variables de entorno.

El contrato EventoNFT.sol - la logica detallada

Es un contrato **ERC-721** (el estandar de NFTs) basado en **OpenZeppelin 4.9.6**. Hereda de ERC721 (la mecanica del token: dueño, transferencias) y de Ownable. Cada boleto es un token unico con un **tokenId** y un dueño.

```
contract EventoNFT is ERC721, Ownable {
    struct EventoMetadata {
        uint    eventoId;    // id del evento en la BD
        string  name;        // nombre del evento
        string  lugar;       // recinto
        string  precio;       // precio en ETH (texto)
        bytes32 hash;        // huella anti-manipulacion
        uint256 timestamp;   // instante de acunacion
    }
    mapping(uint256 => EventoMetadata) public eventoMetadata; // tokenId -> metadata
    mapping(uint256 => uint256)          public eventoToTokenId; // eventoId -> tokenId

    function mintEventoNFT(uint eventoId, string name, string lugar,
                           string precio, bytes32 hash, address to) public returns(uint
256){
        if (eventoToTokenId[eventoId] != 0) revert NFTYaExiste(eventoId);
        if (to == address(0))                revert DireccionInvalida();
        if (bytes(name).length == 0)          revert CampoRequerido('name');
        ...
        uint256 tokenId = _tokenIdCounter++; // 1, 2, 3, ...
        _safeMint(to, tokenId);               // crea el NFT y se lo da a 'to'
        eventoMetadata[tokenId] = EventoMetadata({...});
        eventoToTokenId[eventoId] = tokenId;
        emit NFTMinted(tokenId, eventoId, to);
    }
}
```

Que hace, paso a paso:

- 1 **Valida antes de acunar:** que ese evento no tenga ya un NFT (evita duplicados), que la direccion destino no sea la 0, y que name/lugar no vayan vacios. Si algo falla, revierte con un **error personalizado** (mas barato en gas que un texto).
- 2 **Asigna un tokenId** incremental (empieza en 1; el 0 se reserva para "sin NFT").
- 3 **_safeMint(to, tokenId):** crea el token y lo pone a nombre de la wallet to (el comprador). Aqui nace la **propiedad**.
- 4 **Guarda la metadata on-chain** y el vinculo eventold -> tokenId, y emite el evento **NFTMinted** (para que la blockchain deje rastro).
- 5 **Lecturas:** getTokenId(eventoId), getMetadata(tokenId) y el estandar ownerOf(tokenId) permiten verificar todo despues.

■ **GUION:** *EventoNFT es un contrato ERC-721: cada boleto es un token unico con dueño. Al acunar, primero valida (no duplicar, no direccion cero, no campos vacios), asigna un tokenId, usa `_safeMint` para dar la propiedad al comprador, guarda la metadata dentro del contrato y emite un evento. Con `getTokenId`, `getMetadata` y `ownerOf` cualquiera puede verificar despues que el boleto existe, con que datos, y a quien pertenece.*

Cambios en el BACKEND (mod_02_productos)

Tabla 3.1 - Modificaciones del backend

Archivo	Que es	Que hace
contracts/EventoNFT.sol (nuevo)	Contrato ERC-721	Acuna un boleto NFT por evento con metadata y hash; impide duplicados.
scripts/deploy.cjs	Script de despliegue	Ahora despliega Events + EventoNFT y guarda contract-info.json y nft-info.json .
src/blockchain/eventoNftContract.js (nuevo)	Modulo blockchain NFT (ethers v6)	getNftConfig() (abi+address) y verifyNftComplete() (lee tokenId+metadata+dueno on-chain).
src/service/eventoService.js	Capa de datos	Nuevo metodo updateNftToken(id, tokenId): guarda el tokenId del boleto en la BD.
src/controllers/eventoController.js	Controlador HTTP	Handlers getNftConfig, updateNftToken, verifyNftComplete; la respuesta ahora incluye nftTokenId.
src/routes/eventoRoutes.js	Rutas REST	GET /nft/config GET /nft/verify-complete/:eventid/:address PUT /:id/nft.
src/db/schema.sql	Esquema BD	Nueva columna nft_token_id INT NULL en la tabla eventos.
scripts/verificar-nft.cjs (nuevo)	Prueba automatica	Verifica la logica del NFT en la red Hardhat (mint + lecturas + reglas).
package.json	Dependencias/scripts	@openzeppelin/contracts 4.9.6 + script npm run verify:nft.

Las tres rutas nuevas - para que sirve cada una

Metodo / Ruta	Para que sirve
GET /api/eventos/nft/config	El frontend obtiene el ABI + direccion del contrato NFT para poder acunar firmando con MetaMask.
GET /api/eventos/nft/verify-complete/:eventid/:address	Verificacion de solo lectura: devuelve si el evento tiene NFT, su tokenId, su metadata y si la wallet indicada es la duena.
PUT /api/eventos/:id/nft	Tras acunar en el navegador, guarda el tokenId del boleto en la base de datos.

Detalle de orden de rutas (importante): las rutas /nft/... se declaran **antes** que /:id. Si no, Express interpretaria "nft" como si fuera un id. Es el mismo cuidado que en el lab de facturas.

■ **GUION:** En el backend creamos el contrato, un modulo que habla con la blockchain con ethers, y tres endpoints: uno da la configuracion del contrato al frontend, otro verifica el NFT leyendo la cadena, y el tercero guarda el tokenId en la base de datos tras acunar. Ademas anadimos una columna nft_token_id y un script que prueba el contrato automaticamente.

Cambios en el FRONTEND (frontend_productos)

Tabla 4.1 - Modificaciones del frontend

Archivo	Que es	Que hace
lib/nft.ts (nuevo)	Capa web3 del NFT	mintEventoNFT() firma la acunacion con MetaMask y devuelve {txHash, tokenId}; verificarNFT() consulta al backend.
lib/api.ts	Cliente de la API	getNftConfig() y updateNftToken(id, tokenId).
lib/types.ts	Tipos TypeScript	Nuevo campo nftTokenId en el tipo Evento.
components/dashboard/EventsTable.tsx	Tabla de eventos	Botones ■ Acunar NFT / ✓ Verificar NFT (escritorio y movil); recibe la wallet conectada (account).
app/dashboard/page.tsx	Pagina del dashboard	Pasa la wallet conectada (account) a la tabla.

La logica de acunar (lib/nft.ts) - paso a paso

```
export async function mintEventoNFT(evento, account) {
  const { address, abi } = await cargarNftConfig(); // 1. pide config al backend
  const signer = await new BrowserProvider(window.ethereum).getSigner();
  const contrato = new Contract(address, abi, signer); // 2. contrato con firma
  const hash = solidityPackedKeccak256( // 3. huella de los datos
    ['uint256','string','string'], [evento.id, evento.name, evento.lugar]);
  const tx = await contrato.mintEventoNFT( // 4. FIRMA en MetaMask
    evento.id, evento.name, evento.lugar, String(evento.precio_eth), hash, account)
  ;
  await tx.wait(); // 5. espera minado (Ganache)
  const tokenId = Number(await contrato.getTokenId(evento.id)); // 6. lee tokenId
  return { txHash: tx.hash, tokenId };
}
```

- 1 **Pide la config** del contrato NFT al backend (address + abi) y la cachea.
- 2 **Crea el contrato con un signer** (la wallet del usuario via MetaMask).
- 3 **Calcula el hash** de los datos (keccak256) como prueba anti-manipulacion.
- 4 **Llama a mintEventoNFT**: MetaMask abre el popup para firmar la transaccion.
- 5 **Espera a que se mine** en Ganache y **lee el tokenId** asignado para guardarlo en la BD.

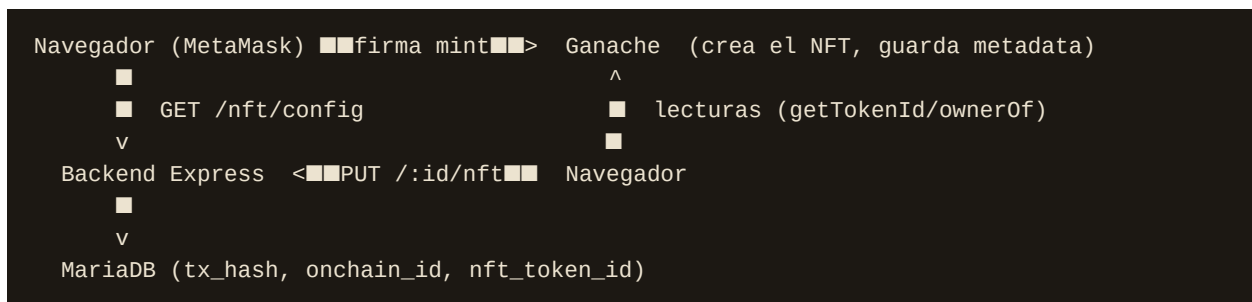
■ **GUION:** En el frontend, al pulsar el boton de acunar, pedimos la config del contrato al backend, creamos el contrato con la wallet del usuario, generamos un hash de los datos, y llamamos a mintEventoNFT: MetaMask pide firmar. Cuando la transaccion se mina en Ganache, leemos el tokenId y lo guardamos en la base de datos. Para verificar, preguntamos al backend, que lee el contrato y nos dice el dueño.

Flujo completo de punta a punta

Que pasa exactamente cuando el usuario acuna y luego verifica un boleto NFT:

Paso	Que ocurre
1. Conectar wallet	El usuario pulsa 'Conectar wallet'. MetaMask entrega su direccion. Sin wallet no se puede acunar.
2. Pulsar ■ Acunar NFT	El frontend pide GET /nft/config al backend (ABI + direccion del contrato EventoNFT).
3. Firmar en MetaMask	Se construye la transaccion mintEventoNFT(...) y MetaMask pide confirmar. El usuario firma.
4. Minado en Ganache	La transaccion se ejecuta en Ganache: se crea el token, se asigna al usuario y se guarda la metadata on-chain.
5. Guardar tokenId	El frontend lee el tokenId y llama a PUT /:id/nft; el backend lo guarda en la columna nft_token_id.
6. Verificar ✓	El frontend llama a GET /nft/verify-complete/:id/:address; el backend lee el contrato (getTokenId, getMetadata, ownerOf) y responde si el boleto existe, sus datos y si te pertenece.

Quien habla con quien



Punto clave: la **acunacion** la firma el navegador directamente contra Ganache (el backend no toca la clave privada del usuario). El backend solo **lee** la cadena para verificar y **guarda** el tokenId. Asi la propiedad del NFT es realmente del usuario.

■ **GUIÓN:** El navegador firma la acunacion directamente contra Ganache, sin pasar la clave privada por nuestro servidor. El backend solo lee la cadena para verificar y guarda el tokenId en la base de datos. Por eso el boleto pertenece de verdad a la wallet del usuario.

Verificacion: debug, errores y prueba de que el NFT funciona

Se reviso toda la integracion y se corrigieron/blindaron los siguientes puntos:

Tabla 6.1 - Revision de bugs y blindajes

Revision	Resultado
Shadowing en el controller (getNftConfig metodo vs import)	Corregido: el import se renombro a readNftConfig para evitar ambigüedad.
Carga de variables de entorno en el modulo NFT	Blindado: eventoNftContract.js importa 'dotenv/config' por si mismo (como su hermano).
Orden de rutas Express (/nft antes de /:id)	Correcto: las rutas /nft/... se declaran antes de /:id.
OpenZeppelin 4.9.6 y _ownerOf disponibles	Verificado en node_modules; el contrato compila (evm target: paris).
Compilacion de contratos (hardhat compile)	OK - 13 archivos Solidity compilados sin errores.
Tipos del frontend (tsc --noEmit)	OK - sin errores de TypeScript.
Carga de modulos backend sin BD ni Ganache	OK - inicializacion perezosa; los modulos importan sin conectar.

Prueba automatica del NFT (npm run verify:nft)

Para demostrar que el **NFT funciona** sin depender de Ganache durante la revision, se ejecuta el contrato en la **red en memoria de Hardhat** (la misma logica que correra en Ganache). Resultado real de la ejecucion:

```
EventoNFT desplegado en: 0x5FbDB2315678afecb367f032d93F642f64180aa3
mintEventoNFT OK tx: 0x990977...c6a2a16
  evento NFTMinted -> tokenId: 1 eventoId: 1
  getTokenId      -> 1
  ownerOf         -> 0x7099...79C8 (comprador OK)
  getMetadata     -> name: Rock de los 90s | lugar: Coliseo Nacional | precio: 0.08
  getTokenId(999) -> 0 (evento sin NFT OK)
regla NFTYaExiste OK (no se acuna 2 veces el mismo evento)
regla CampoRequerido OK (name/lugar no pueden ir vacios)
regla DireccionInvalida OK (no se acuna a la direccion 0)
==> TODAS LAS COMPROBACIONES DEL NFT PASARON CORRECTAMENTE
```

Conclusion: el contrato acuna, asigna propiedad, guarda metadata y aplica sus reglas de error correctamente. Al desplegarlo en Ganache con `npm run deploy` se comporta identico, y el dashboard permite acunar y verificar boletos NFT reales.

■ **GUION:** Revisamos toda la integracion, corregimos un shadowing en el controller y blindamos la carga de variables de entorno. Para demostrar que el NFT funciona, ejecutamos el contrato en la red de Hardhat en segundo plano: acuna, asigna el dueño, guarda la metadata y respeta todas las reglas. En Ganache se comporta igual.

Como ponerlo en marcha (con Ganache)

```
# BACKEND (mod_02_productos)
npm install           # trae @openzeppelin/contracts
npm run compile       # compila Events.sol + EventNFT.sol
# migrar BD (una vez): ALTER TABLE eventos ADD COLUMN nft_token_id INT NULL;
# (Ganache GUI abierto en http://127.0.0.1:7545, private key en .env)
npm run deploy        # despliega AMBOS contratos en Ganache
npm run verify:nft    # (opcional) prueba la logica del NFT en Hardhat
npm run dev           # API en http://localhost:3000

# FRONTEND (frontend_productos)
# pega la direccion de Events en .env.local -> NEXT_PUBLIC_CONTRACT_ADDRESS
npm run dev           # dashboard en http://localhost:3001
```

En el dashboard: **Conectar wallet** ->  en un evento -> **firmar en MetaMask** -> el boton pasa a **✓ NFT #N** -> pulsar **✓** para verificar el dueño on-chain.

Glosario para la exposicion

Termino	Significado simple
ERC-721	Estandar de NFTs: cada token es unico y tiene un dueño.
Mint / Acunar	Crear un NFT nuevo y asignarlo a una wallet.
tokenId	Identificador unico del NFT dentro del contrato (empieza en 1).
OpenZeppelin	Libreria de contratos seguros y auditados (base ERC-721).
Metadata on-chain	Datos del boleto guardados en el propio contrato.
hash (keccak256)	Huella de los datos; prueba de que no fueron alterados.
MetaMask	Billetera del navegador que firma transacciones.
Ganache	Blockchain local de pruebas (10 cuentas con 100 ETH).
Hardhat	Herramienta (en segundo plano) para compilar, desplegar y probar.
ownerOf(tokenId)	Funcion estandar que devuelve el dueño de un NFT.

Fin del guion - Integracion del Boleto NFT (ERC-721) | mod_02_productos + frontend_productos